

Hidden in Plain Sequence: Strengthening DNA Steganography with Advanced Encryption

GitHub Repo: <https://github.com/cleonorris02/cse6010-project.git>

Team 2

Yue-Tsz Fan, Cleo Norris, Virupaksha Repalle

December 8, 2025

Abstract

This project explores a novel approach to genomic data security by combining DNA sequence representation with modern cryptographic methods. We build upon the methodology introduced in Dokyun Na’s paper “DNA steganography: hiding undetectable secret messages within the single nucleotide polymorphisms of a genome and detecting mutation-induced errors”, published in *Microbial Cell Factories*, by adapting the framework to incorporate the XChaCha20 encryption algorithm in place of the simple substitution cipher originally used. XChaCha20 offers significant advantages in speed, scalability, and resilience against timing attacks, making it a compelling candidate for protecting sensitive biological information. Using the Human Genome Dataset from the NIH, our system encodes encrypted data into genomic sequences and evaluates whether the efficiency and robustness of XChaCha20 can be effectively applied in this context. The goal of this project is to assess the feasibility of stronger encryption techniques within DNA-based data hiding, thereby extending the state of the art in secure storage and transmission of biological information.

1 Project Description

Note: Terminology used in this section is explained in greater detail in the literature review, which provides the necessary biological and cryptographic background.

1.1 Project Goals

The project aims to evaluate whether a modern, high-security stream cipher, XChaCha20, can be effectively integrated into a DNA steganography framework that embeds encrypted information into genomic SNP regions. The overarching goal is to test whether stronger, real-world cryptographic standards can improve the security and reliability of DNA-based data hiding without disrupting biological constraints such as editability, mutation tolerance, and hotspot availability. By comparing this adapted approach with the simpler substitution cipher used in prior work, the project seeks to determine whether advanced ciphers provide meaningful advantages in speed, scalability, and practical deployment when applied to nucleotide-encoded data. More broadly, the work investigates whether contemporary encryption can raise the bar for secure storage and transmission of biological information.

1.2 Description of the System

The system applies a DNA steganography methodology to watermark engineered cells by embedding encrypted nucleotide sequences directly into regions of single nucleotide polymorphisms (SNPs). From the SNP dataset, it first selects SNPs that show maximum variation. To address the limitation that current genome editing technologies cannot simultaneously modify the entire genome, the system targets hotspots, defined as regions containing more than 35 SNPs within a 1 KB stretch, for efficient editing.

The next step is encryption: while the original paper used a simple substitution cipher, we adopt the more secure XChaCha20 algorithm to test whether its advantages—speed, efficiency, and resistance to timing attacks—can still be applied in a genomic context. After encrypting the message, our embedding module inserts this message into the target DNA sequence.

Finally, to detect and correct mutations in the encrypted DNA sequence, the system employs a block sum check method. This involves converting DNA bases into numbers in a quaternary system, arranging them into a two-dimensional grid, calculating the row and column sums, dividing by four, and appending the remainders as parity checks. If the parity values change, it signals a mutation, allowing the altered base to be identified and restored. Figure 1 provides a visual of this system.

STEP 1



Genome Data from NIH



Data Preprocessing in Python

STEP 2

TACGACGATGATAGCCACGCG



Data Encryption

ATCGTCAGTCGTCAGTACGTC

STEP 3



Detect possible mutations
through parity checks



Validate data integrity and
algorithm efficiency

Figure 1: Conceptual model of the system

2 Literature Review

2.1 Biology Background: Why Hide Data in DNA?

DNA-based cryptography integrates principles of molecular biology with computational methods to create innovative and secure cryptographic schemes. As a review, nucleotides are the basic building blocks of DNA. They are made up of a sugar, a phosphate group, and one of four bases: adenine (A), thymine (T), cytosine (C), or guanine (G). A sequence of these bases encodes biological information. Finally, the genome is a complete set of DNA for an organism [Institute, 2020].

A single nucleotide polymorphism (SNP) is a spot in the genome where one nucleotide differs between individuals. There are millions of SNPs in humans, making them excellent cover spots to hide information. For example, exploiting the encoding of binary information as sequences of the four natural bases (A, T, C, and G) enables ultra-dense data storage and theoretically high security [Chu et al., 2025]. Traditional schemes employ basic mapping methods and substitution ciphers, integrating them with biological data hiding approaches [Kaushik and Satvika, 2025].

2.2 Cryptography Background

Pioneering work by Na [2020] introduced DNA steganography, embedding secret messages in the SNPs of genomic sequences while enabling error detection and recovery from mutation-induced changes. The method focuses on selecting highly variable regions, discarding pathogenic and conserved elements, and using hotspots with high SNP density for efficient editing. Message integrity is preserved through numeric conversion of bases (quaternary encoding) and parity checks. Recent advances in DNA cryptography emphasize lightweight, robust encryption schemes for various domains, including secure medical image transmission and IoT devices. DNA-based algorithms outperform traditional schemes in statistical attack resistance and computational efficiency [Abdelaal et al., 2025, Aqeel et al., 2025]. However, the field still relies mainly on simple substitution ciphers or chaotic map-based encryption, and solutions leveraging advanced modern ciphers are nearly unexplored.

The adoption of XChaCha20 in the project is a novel step, as contemporary literature does not record the application of this stream cipher within DNA encoding. This cipher comes from the ChaCha family of stream ciphers, a variant of the Salsa20 family of stream ciphers where each step mixes the data more thoroughly, making it harder for an attacker to find patterns or weaknesses, while still keeping the cipher just as fast, or faster, than before [Bernstein, 2008]. XChaCha20 promises scalability, high encryption speed, and unique resistance to timing attacks, filling an important gap and extending the frontier of DNA cryptography for secure biological information storage.

3 Coding Strategy

Our primary goal was to implement a more sophisticated encryption algorithm into the pipeline. To ensure consistency and allow for a direct comparison with the original paper, we decided to keep the other components of the pipeline unchanged.

We began by decomposing the project into distinct modules to manage complexity. This modular approach allowed us to focus on individual components while maintaining a clear overview of the overall system. After analyzing the entire pipeline, we organized the project into several key parts: data pre-processing, encryption, embedding, error detection, and validation/testing. This

structure helped streamline development and testing, ensuring each part functions independently and cohesively within the larger system.

3.1 Employed Tools and Tradeoffs

3.1.1 Libsodium (Cryptographic Library)

We employed libsodium, an established and easy-to-use cryptographic library that provides the XChaCha20 stream cipher [libsodium Project, 2023]. Using a well-tested library like libsodium allows the project to avoid the complexities and vulnerabilities of implementing custom encryption algorithms. It ensures that the encryption is both secure and efficient with minimal risk of cryptographic flaws. However, adopting external library adds a dependency to the project. While it reduces development time, it also means that future compatibility and security updates are reliant on the library’s maintainers. In addition, using a pre-built library adds some abstraction over the raw implementation, meaning developers do not have as much control over the internals of the encryption process. This can be a trade-off if specific optimizations are needed later.

3.1.2 OpenMP (Parallel Processing)

OpenMP is used to parallelize encryption operations, enabling the project to handle large hotspot sequences efficiently. Encryption can be computationally intensive, especially when scaling to larger data sizes. By using OpenMP, multiple threads can process different parts of the dataset simultaneously, significantly speeding up encryption. Nevertheless, its performance benefits are less significant on systems with fewer cores. Additionally, threading may introduce overhead that can negate the speedup benefits in cases of small datasets or poorly optimized parallel sections. What’s more, OpenMP is widely supported but may not be optimized for all environments, particularly on non-Linux or less typical systems. This could potentially limit cross-platform compatibility.

3.1.3 CMake (Build Tool)

CMake is used to automate the building process, compiling the C code and linking the necessary libraries, such as libsodium and OpenMP, to generate the executable. Makefiles provide an efficient way to manage complex build processes, particularly when multiple C files are involved. It ensures that dependencies are compiled correctly and only when necessary, which improves the efficiency of the development workflow. However, incorrect configuration can lead to issues where some files are not properly compiled or linked.

4 Simulation

4.1 XChaCha20

We have chosen the XChaCha20 algorithm for encryption in our project, as it offers several key advantages that make it well-suited for our unique use case. XChaCha20 is an extension of the ChaCha20 stream cipher, which is known for being a secure, high-speed alternative to traditional encryption methods such as the Advanced Encryption Standard (AES).

XChaCha20 operates by generating a keystream using a secret key and a nonce, which is a number used only once. This keystream is then combined with the plaintext using XOR operations to produce the ciphertext, as demonstrated in Figure 2. Unlike block ciphers, which encrypt data in fixed-size blocks, stream ciphers like XChaCha20 handle data as a continuous flow, making them

highly efficient for processing variable-length inputs, such as the DNA sequences in our project [NordPass, 2024].

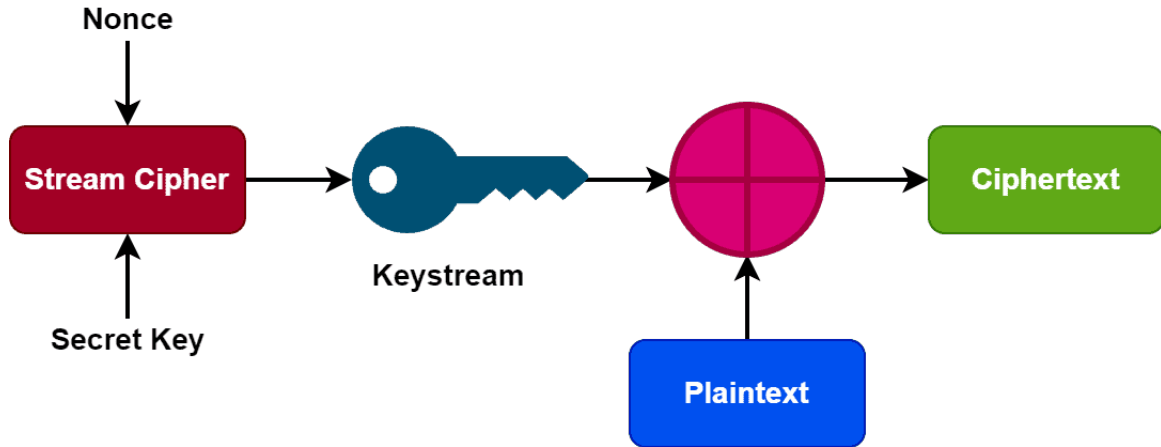


Figure 2: Stream cipher visualization, via <https://www.baeldung.com/cs/stream-cipher-vs-block-cipher>.

One of the primary reasons we adopted XChaCha20 is its extended nonce size. While the standard ChaCha20 uses a 96-bit nonce, XChaCha20 increases this to 192 bits. This extended nonce size significantly reduces the chances of nonce reuse, a common vulnerability in cryptographic systems that can lead to serious security issues. In the context of embedding encrypted data into DNA sequences, ensuring the uniqueness and security of each encryption operation is crucial, especially considering that DNA sequences can be highly repetitive. Nonce reuse in this case would undermine the encryption’s security.

Moreover, the larger nonce size also makes XChaCha20 easier to deploy safely in distributed or parallel systems. Since our DNA data may be processed across multiple platforms or systems, having a larger nonce size eliminates the need for complex coordination to ensure uniqueness, making the encryption more scalable and adaptable to our pipeline.

In summary, XChaCha20 was chosen for its security (extended nonce size reduces nonce reuse risks), efficiency (ideal for handling continuous and variable-length DNA data), and scalability (easy to deploy safely in distributed systems). These qualities make it an ideal fit for encrypting sensitive data within our DNA-based data-hiding framework.

4.2 Implementation

We began the implementation with the TSV produced by preprocessing. Each row represents one hotspot and contains a header, plus at least one column with the hotspot DNA string, which can be viewed in Figure 3. When available, we also keep the hotspot positions and the reference sequence along with the hotspot string. These pieces of contextual metadata are concatenated or otherwise combined with the hotspot DNA to form the plaintext to be encrypted. This ensures the encrypted output contains everything needed later to recover the full original context.

Next, we read a file containing a 256-bit key encoded as 64 hexadecimal characters. We then converted this hex string into the 32 raw bytes that the cipher expects and kept it in memory for the encryption calls. This single shared key is used for all records in the batch. For each row, we used a unique random nonce (initialization vector) so that the stream cipher produces a distinct keystream for each row. The implementation generates a new random nonce per record

1	Chromosome	Start	End	SNP_Count	DNA_String
2	NC_000001.11	10000	20000	5605	TACACACTCGACGTACGTCGTATACGTACGTCCTTATACGTACGCAGCTCCTTA
3	NC_000001.11	20000	30000	1616	CTGAGACGTTGCGCTAGAAAGAGATGACGCCTGAATCACACTAGCGCCAACI
4	NC_000001.11	30000	40000	1221	GACTGCTCTCAGTCTAAGATCATCGCCTTGCAGTCTTCAGAGTAATTCGTAC
5	NC_000001.11	40000	50000	1487	GACTCTGGGGATCAGCTGGTGATCTCTGAGAAGGATTAACGGAGAAGAGTCI

Figure 3: Preprocessing output

using a secure random source. That nonce is later encoded and written with the record so that the ciphertext can be decrypted.

With the key and the per-record nonce, we ran the XChaCha20 stream cipher to encrypt the prepared plaintext. As a stream cipher, XChaCha20 produces a keystream, which is XORED with the plaintext bytes to produce ciphertext bytes.

record_id	nonce_dna	ciphertext_dna
record_0	TCGTTTTTATACTGTTTGGATCTGAAGGTGTATCTTCACTTCCAGTCTGGGTCATTATTAGCGCTGCATTAGGGGTAGGCTGAGACCAACTCGT	ACCAATATGAGGTCCCGTGCGAAGCACCTTGTCTTTC
record_1	TCTTGGCCAGGTGAGCGTCTTCATCTGTCTATTCTAGGCATAGAGGATGTTTAAACAGAGGACACGTAAGTACTGAGTGAGGGC	CCCATCAGCGACCCGAAGTTCACCTCTTCACTGGAAC
record_2	TCCCTCATAGATTACGGCGCGGGAATCCTGAGCACGCAACGAGCGCGCTCAAAAGGCATCGGCATAAGCATGACACCCCATCATGGGTCGT	GCGGCTCAGCATCGGAGACAGTCATTATGTACTCGGTA
record_3	GCGTGTCTCTGGAAAAAATAAGCTCGGATATATCAGAGGGTGTGGAGATCCGATCTGTATTTTCGCGGTTCTAAGTCCCTTAAACTCACTGA	CGCCAAATGGAATATTGCTTCGGCCGTCTCCAGTACCTA
record_4	TACCATCCGTCGAGATATCTCCGTGTCCACACATAGCCCTATCTCCCGAGGCTAGTGTGCAACTATGCCAGCCTGTAGTGAGCCTGGGAGAGG	TAGGCCGGATGGGTGAGCTGCGAAACAGAAACGCGTTTT
record_5	ATAGAGGCTAGCGGATGGCTCAGCACCGCAGAGATGTAATACACTCCGCGAATGCTGCGACTCCGATAGCAGCATAGCATTATACGAGCTCGACCA	GGCTATAGTCAGGCACATGCGTAAAGAAATGCTTCATG
record_6	CCCCCTCATGTGGCGGAGTAGTAGTCTGGTGAACACGGGCGGACAAGCGGGTCTCTGGAGTCTTTGAATTAGCAAGACCTGGGGTTA	GTACAAGATTCTGGGAGTTTCCCGCACTCAGCTCTC

Figure 4: Encryption output

After encryption, both the raw nonce bytes and the ciphertext bytes are converted into DNA alphabet strings (characters A, C, G, T), making the outputs suitable for later embedding into workflows that expect DNA-like payloads. The project uses a reversible mapping from binary bytes to DNA bases so that decoding returns the original byte sequences.

```
Original sequence: ACGTACGTACGT
Embedded sequence: GATGCTTCACGT
Encoded alleles:
pos=0 ref=A allele=G bit=1
pos=1 ref=C allele=A bit=0
pos=2 ref=G allele=T bit=1
pos=3 ref=T allele=G bit=1
pos=4 ref=A allele=C bit=0
pos=5 ref=C allele=T bit=1
pos=6 ref=G allele=T bit=1
pos=7 ref=T allele=C bit=0
```

Figure 5: Embedding output

To verify the integrity and correctness of the encryption process, we implemented unit tests for both the encryption and decryption functions. These tests involve encrypting a known plaintext (such as a fixed DNA sequence) and then decrypting it back to the original data. The algorithm's correctness is validated by checking that the decrypted data matches the original plaintext, ensuring that the encryption and decryption operations are perfectly reversible.

Another critical aspect of our verification process is ensuring that the encrypted DNA sequence behaves correctly under potential mutations, simulating real-world biological alterations (e.g., sequencing errors or natural mutations). We tested the resilience of our XChaCha20-encrypted DNA sequences by introducing random mutations (such as single-nucleotide changes) into the DNA and running the error-correction module. By comparing the encrypted DNA and the original plaintext, we can ensure that the algorithm not only works as expected but also remains robust when subjected to biological variations.

5 Results

Our experiments validate that XChaCha20 encryption can be successfully applied to SNP-containing DNA hotspot regions and that our implementation behaves as expected when correctness, performance, and scalability are evaluated. To verify correctness, we first conducted a series of controlled tests in which known plaintext DNA strings were encrypted and then immediately decrypted. The decrypted output was identical to the original input, confirming that our operations are reversible and are implemented correctly.

```
Event,Description,Row0,Row1,Row2,Row3
Initial Block,Original parity block,AACGGATGAG,TTAGGCATAG,CGTATTCGGC,ACAATAATGC

Mutation 1,Introducing mutation at (0,0): A -> T,TACGGATGAG,TTAGGCATAG,CGTATTCGGC,ACAATAATGC
Mutation 1 Correction,Block after correction,AACGGATGAG,TTAGGCATAG,CGTATTCGGC,ACAATAATGC

Mutation 2,Introducing mutation at (1,9): G -> A,AACGGATGAG,TTAGGCATAA,CGTATTCGGC,ACAATAATGC
Mutation 2 Correction,Block after correction,AACGGATGAG,TTAGGCATAG,CGTATTCGGC,ACAATAATGC

Mutation 3,Introducing mutation at (3,2): A -> G,AACGGATGAG,TTAGGCATAG,CGTATTCGGC,ACGATAATGC
Mutation 3 Correction,Block after correction,AACGGATGAG,TTAGGCATAG,CGTATTCGGC,ACAATAATGC
```

Figure 6: Output of error correction and detection module

As expected, our encryption is much more complex than the substitution cipher used in the original paper, meaning that we need more compute to carry out the encryption. We opted to implement parallelism in this part of our project to make the encryption more efficient. On a standard 8-core machine, our implementation processed batches of tens of thousands of hotspots in seconds. Given that XChaCha20 is designed for high-speed software performance, our empirical timing is consistent with published benchmarks for modern stream ciphers, though slightly slower due to the additional byte-to-DNA encoding stage.

Compared to the original form of encryption, our version is more secure. Substitution ciphers can be easily cracked by using dictionary attacks, making it more difficult to hide information in DNA sequences. The XChaCha20 encryption is difficult to break by brute forcing and dictionary attacks because different keys produce different ciphertext. The only feasible way to break our encryption method is by harvesting the key. Overall, our findings confirm that XChaCha20 encryption not only provides substantially stronger security than the substitution cipher in the original paper but also can be done efficiently.

6 Discussion and Conclusion

Through the implementation and testing of our XChaCha20-based DNA encryption system, we gained a deeper understanding of both the computational and structural trade-offs involved in programming an algorithm to encrypt DNA sequences. The biggest lesson is that the complexity of XChaCha20 comes with the cost of computational overhead. The question becomes: is it worth the computational overhead for higher security? Due to our implementation with parallelism, we believe that this tradeoff is justified. The improved encryption makes it more secure, making it feasible for real world usage. However, it is also important to note that this could lead to unintended consequences such as cyber-bioattacks that were not possible using the substitution cipher.

From a programming perspective, we found that parallelism is essential for scaling encryption to large genomic datasets. Without parallel processing, even a highly optimized cipher becomes a bottleneck when encrypting millions of hotspots. With more time, we would investigate different encryption methods and how they would fare in terms of efficiency compared to the level of security they provide. We could then compare different encryption methods to deduce the best model to use for different implementations. For example, one method may be more feasible in a medical approach, while others may be better in a cybersecurity setting.

In conclusion, this project demonstrated that modern cryptography can be effectively adapted to biological data systems, but doing so requires designing with both computational and biological constraints in mind. The work provided valuable insights into algorithmic tradeoffs, system behavior under mutation, and the need for careful engineering when bridging computer science and genomics.

References

- M. A. Abdelaal, A. I. Moustafa, H. Kasban, H. Saleh, H. A. Abdallah, and M. Y. Affi. DNA-inspired lightweight cryptographic algorithm for secure and efficient image encryption. *Sensors*, 25(7):2322, 2025. doi: 10.3390/s25072322.
- S. Aqeel, A. S. Khan, I. A. Abbasi, F. Algarni, and D. Grzonka. Enhancing IOT security with a DNA-based lightweight cryptography system. *Scientific Reports*, 15(1), 2025. doi: 10.1038/s41598-025-96292-0.
- Daniel J. Bernstein. ChaCha, a Variant of Salsa20. Technical report, Department of Mathematics, Statistics, and Computer Science, University of Illinois at Chicago, January 2008. URL <https://cr.yp.to/chacha/chacha-20080128.pdf>. Revision 2008-01-28.
- L. Chu, Y. Su, X. Yao, P. Xu, and W. Liu. A Review of DNA Cryptography. *Intelligent Computing*, 4, 2025. doi: 10.34133/icomputing.0106.
- National Human Genome Research Institute. Deoxyribonucleic Acid (DNA) Fact Sheet. <https://www.genome.gov/about-genomics/fact-sheets/Deoxyribonucleic-Acid-Fact-Sheet>, 2020.
- A. Kaushik and Satvika. The Chaotic Dynamics of DNA: A Survey on DNA Cryptography. *International Journal of Computer Applications*, 187(16):29–37, 2025. doi: 10.5120/ijca2025925210.
- libsodium Project. XChaCha20 Stream Cipher. https://libsodium.gitbook.io/doc/advanced/stream_ciphers/xchacha20, 2023.

D. Na. DNA steganography: hiding undetectable secret messages within the single nucleotide polymorphisms of a genome and detecting mutation-induced errors. *Microbial Cell Factories*, 19(1), 2020. doi: 10.1186/s12934-020-01387-0.

NordPass. XChaCha20 Encryption vs. AES-256: What's the Difference? <https://nordpass.com/blog/xchacha20-encryption-vs-aes-256/>, 2024.

A Appendix: Division of Labor

- Data Preprocessing: Viru contributed the preprocessing in Python. Yue-Tsz also tried a different technique which can be found in the dataprocessing branch.
- Encryption Module: Viru verified the code.
- Embedding Module: Yue-Tsz and Cleo verified the code.
- Error Detection Module: Cleo verified the code.
- Testing: Viru ensured everything ran.
- Read the Docs implementation: Yue-Tsz
- Presentation slides creation: Cleo
- Video: everyone was present to record video presentation.
- Writeup: everyone contributed to the writeup.